SOFTWARE PRODUCT · DOCUMENTATION

Lalaba

Marketplace

A FINDXNY client project: a multi-role laundry marketplace connecting customers, laundromats, home washers, staff, fulfillment operations, and platform administration.

ROLE	PERIOD	PLATFORMS
Junior Full-Stack Engineer	**Oct 2025 – Aug 2026**	**Web · Mobile · Backend**

CONTENTS

1. Overview
2. Scope and personal contribution
3. Product surfaces
4. System architecture
5. Technology stack
6. Capabilities
7. Engineering deep dives
8. Repositories
9. Engineering decisions
10. System scale
11. Evidence, status and limitations

Prince John Gandollas · Full-Stack Engineer
pgandollas@gmail.com · LinkedIn · GitHub · pjdevph.vercel.app

Generated from the portfolio's content on 2026-10-17. Live version: pjdevph.vercel.app/projects/lalaba

1. Overview

A FINDXNY client project: a multi-role laundry marketplace connecting customers, laundromats, home washers, staff, fulfillment operations, and platform administration.

PROJECT	Lalaba
TYPE	Marketplace
ROLE	Junior Full-Stack Engineer
PERIOD	Oct 2025 – Aug 2026
PLATFORMS	Web · Mobile · Backend
TEAM	Built as part of a small engineering team

2. Scope and personal contribution

> Counts such as "115 Edge Functions" describe the whole platform, not only Prince's own work. His own contributions are the areas of ownership listed in this section.

- Branch-scoped device approval
- Wallet webhook idempotency
- Server-authoritative pricing
- Permission/role enforcement

What was built, in detail:

- Built out a NestJS + Apollo GraphQL backend (187 module directories, 57 resolvers, 68 Mongoose schemas) shared by both mobile apps and the admin panel, as part of a small engineering team.
- Implemented branch-scoped staff device approval — single-active-session gating, a 5-minute cached auth TTL, and per-branch grants enforced server-side, not just hidden in the UI.
- Built server-authoritative platform-fee, booking-availability, and booking-policy modules so pricing and capacity rules can't be recomputed client-side.
- Implemented the Xendit wallet top-up webhook: timing-safe callback-token verification, amount/currency validation against the stored intent, and a uniquely-indexed ledger write so a retried callback can never double-credit.

3. Product surfaces

Customer App

`Expo · React Native`

The app end customers use to find laundry providers, book pickup/delivery, and pay from an in-app wallet.

- Firebase Auth (JS SDK) + App Check
- Google Maps for provider discovery
- Custom fetch-based GraphQL client — no Apollo/codegen

Partner App

`Expo · React Native`

The app laundromats, home washers, and staff use — bookings, verification, and device-gated access.

- Camera + face detection + biometrics for verification/KYC
- Branch-scoped staff device approval
- QR-code support for pickup/handoff flows

Admin Panel

`Next.js`

Internal operations console — 29 feature routes mirroring the backend's own module boundaries.

- Merchants, orders, wallets, promotions, campaigns, booking-policy
- Hand-written GraphQL query/mutation layer (no client library)
- Role/permission management UI

Marketing Website

`Next.js`

A standalone public-facing site — intentionally has no live connection to the backend.

- Static marketing content only
- No auth, API, or payment logic
- Separate repo, separate deploy

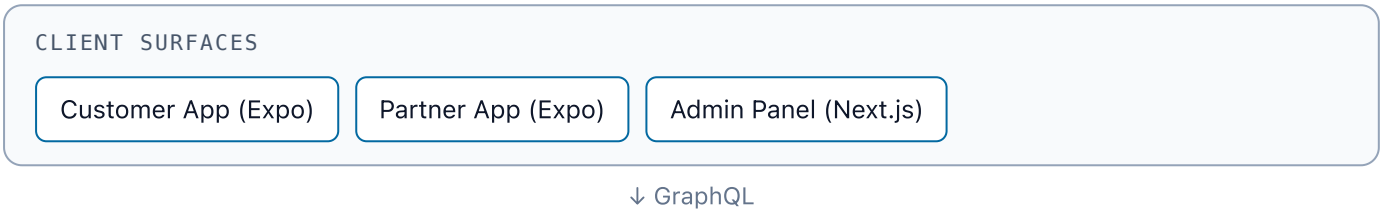
Backend Platform

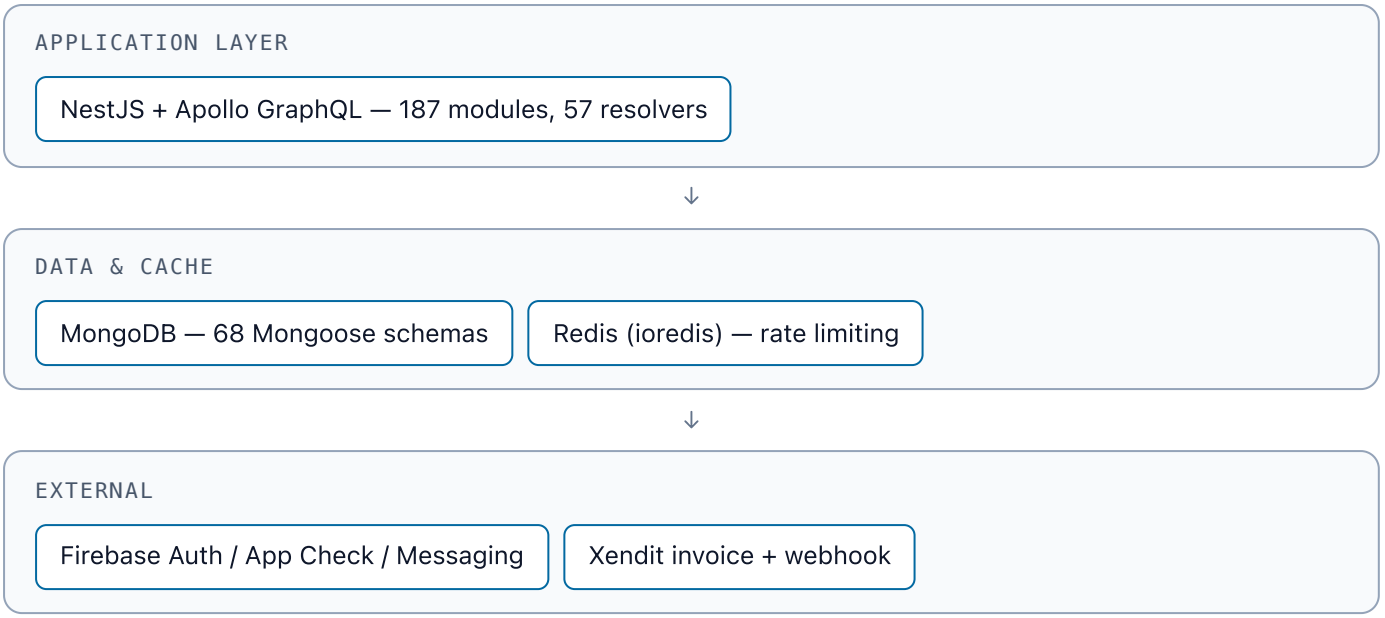
`NestJS · Apollo GraphQL`

The shared API every client talks to — one backend behind both mobile apps and the admin panel.

- 187 module directories, 57 GraphQL resolvers, 68 Mongoose schemas
- MongoDB + Redis (rate limiting/caching)
- Xendit wallet webhook, Firebase Admin SDK

4. System architecture





> The marketing website is deliberately unwired: its own repository, with no API, auth or payment connection to the platform above.

5. Technology stack

Area	Technologies
Mobile	Expo (React Native 0.83), expo-router, react-native-maps
Web	Next.js (admin + website), React 19
Backend	NestJS 11, Apollo Server / GraphQL, Mongoose 9
Data & Cache	MongoDB, Redis (ioredis)
Auth & Trust	Firebase Auth (JS SDK), Firebase App Check, Firebase Cloud Messaging
Payments	Xendit (invoice + webhook)
Verification	react-native-vision-camera, Face detection, Biometrics
Delivery	EAS Build/Update (dev/preview/production)

6. Capabilities

Customer

- Provider discovery on a map, booking, and order tracking
- In-app wallet with Xendit-backed top-ups
- Push notifications via Firebase Cloud Messaging

Partner / Provider

- Branch-scoped staff device registration and approval
- Camera-based verification (face detection, biometrics) for KYC
- Booking, POS, and online-order handling modules

Platform & Access

- Server-authoritative platform-fee, booking-availability, and booking-policy modules
- Permission/role modules enforced at the API layer, not just the UI
- Promotions and voucher redemption as first-class backend modules

Admin Operations

- 29 admin routes mirroring backend module boundaries (merchants, orders, wallets, promotions, campaigns)
- Booking-policy and platform-fee configuration UI
- Wallet and KYC review tooling

7. Engineering deep dives

Staff device approval: how access is gated per branch

1. A staff device registers itself against a specific branch
2. An owner/admin approves or blocks it — status lives on the device record, not the client
3. Each authorization check is cached for 5 minutes to avoid hammering the DB on every request
4. A device is gated to a single active session — deactivating it can't be silently bypassed by the app still holding an old token
5. A backfill routine (onModuleInit) normalizes devices created before the status/session fields existed, so old rows don't fall through the gate unprotected

Wallet top-up: how a payment callback credits exactly once

1. Client requests a wallet top-up; a pending top-up intent is recorded with its own external ID
2. Xendit hosts the actual invoice/payment collection
3. Xendit calls back to POST /webhooks/xendit with an x-callback-token header
4. The token is compared with SHA-256 + a timing-safe comparison — no early-exit string compare that could leak timing information
5. On PAID/SETTLED, the amount and currency are validated against the stored intent before anything is credited

6. The credit itself goes through one transactional path guarded by a unique ledger index, so a retried Xendit callback returns "already posted" instead of crediting twice

8. Repositories

Public, redacted showcases of each part of the system:

Part	Repository
Backend	https://github.com/pjDevph/lalaba-backend-showcase
Customer app	https://github.com/pjDevph/lalaba-customer-showcase
Partner app	https://github.com/pjDevph/lalaba-partner-showcase
Admin panel	https://github.com/pjDevph/lalaba-admin-showcase
Website	https://github.com/pjDevph/lalaba-website-showcase

9. Engineering decisions

Server-authoritative pricing

PROBLEM

Different clients could otherwise compute inconsistent totals, platform fees, and booking capacity.

DECISION

Centralize fee, voucher, availability, and booking-policy rules in dedicated backend modules (platform-fee, booking-availability, booking-policy).

IMPACT

The backend is the single source of truth for money and capacity — no client can under- or over-charge by recomputing a total locally.

Branch-scoped device approval

PROBLEM

Staff devices and permissions differ by branch, and a lost/shared device is a real access risk in a laundry-branch setting.

DECISION

Gate each device to a single active session server-side, cache the auth check for 5 minutes, and scope grants to the approved branch.

IMPACT

A device can't silently stay authorized after being deactivated, and permissions can't be inferred from UI state alone.

10. System scale

- 5 independent repos
- 187 backend modules
- 57 GraphQL resolvers

- 68 Mongoose schemas
- Multi-env EAS

> These figures describe the whole system. His own contributions are the areas of ownership listed in this section.

11. Evidence, status and limitations

- This document describes architecture and behaviour only. It contains no credentials, internal endpoints or customer data.
- No performance, revenue or user-count figures are published for this project.
- The system is a small-team project; the system counts above describe the whole backend.

Sources:

- Case study: https://pjdevph.vercel.app/projects/lalaba
- Backend: https://github.com/pjDevph/lalaba-backend-showcase
- Customer app: https://github.com/pjDevph/lalaba-customer-showcase
- Partner app: https://github.com/pjDevph/lalaba-partner-showcase
- Admin panel: https://github.com/pjDevph/lalaba-admin-showcase
- Website: https://github.com/pjDevph/lalaba-website-showcase